In [ ]:
```
# Initialize Otter
import otter
grader = otter.Notebook("practice07.ipynb")
```

Student names and e-mails:

YOUR NAME — your@calvin.edu

YOUR NAME — your@calvin.edu

Practice 07 — Who Gets Adopted? Scaling, Encoding, Pipelines and Missing Data

An animal shelter records every animal that comes in, and whether it was adopted within 30 days. The shelter would like to know early which animals are likely to wait, so volunteers can give them extra attention. Can kNN learn that from the records?

Not as they are. The records mix dollars with years, words with numbers, and some boxes were left blank. This week's work is getting the data ready.

SLO	Description
09A	Apply preprocessing steps (scaling and one-hot encoding) and explain why each is needed before modeling
09B	Build a scikit-learn pipeline that chains preprocessing and modeling steps into a single reproducible workflow
09C	Identify how missing data arise (MCAR, MAR, MNAR) and choose appropriate imputation strategies for each case

---

The Dataset: 2,000 Shelter Animals

The data is **simulated**. Its patterns were put there on purpose, and at the end you'll see exactly how, including the true values hidden behind every blank. With real data, nobody can show you those. That makes it a good place to practice, but nothing here is a claim about real shelters.

Column	What it holds
`animal_id`	the shelter's ID for the animal
`species`	Dog, Cat or Rabbit
`size`	Small, Medium, Large or Extra large
`intake_type`	how the animal arrived: **Stray** (found), **Owner surrender** (given up by its owner) or **Transfer** (sent by another shelter)
`age_years`	age, in years
`weight_kg`	weight at intake, in kg
`behavior_score`	temperament assessment, from 1 (very stressed or reactive) to 10 (calm and friendly)
`adoption_fee`	in dollars
`adopted`	**the label**: 1 = adopted within 30 days, 0 = not

Setup (given — just run it)

In [ ]:
```
import numpy as np
import pandas as pd
import plotly.express as px
from sklearn.model_selection import train_test_split, cross_val_score, GridSearchCV
from sklearn.neighbors import KNeighborsClassifier
from sklearn.preprocessing import StandardScaler, OneHotEncoder, OrdinalEncoder
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline, make_pipeline
from sklearn.impute import SimpleImputer, KNNImputer

shelter = pd.read_csv("https://cs.calvin.edu/courses/data/202/26fa/datasets/shelter.csv")
print(len(shelter), "animals | adopted within 30 days:", round(shelter["adopted"].mean(), 3))
shelter.head()
```

Three columns have blanks. As in Monday's class, Parts 1–3 use only the animals with **nothing blank**, and Part 4 brings back the rest.

The features are the four numeric columns, `size`, and the two word columns. The split is given: 25% for testing, with `stratify=y`. **The test set stays locked** until a step says to use it.

In [ ]:
```
numeric = ["age_years", "weight_kg", "behavior_score", "adoption_fee"]
words = ["species", "intake_type"]
sizes = ["Small", "Medium", "Large", "Extra large"]          # smallest to largest

complete = shelter.dropna()                                  # Parts 1-3: the animals with no blanks
X = complete[numeric + ["size"] + words]
y = complete["adopted"]

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.25, stratify=y, random_state=1)
print(len(complete), "animals |", len(X_train), "training |", len(X_test), "test")
```

Part 1 — Dollars vs. Points: Scaling (SLO 09A)

In [ ]:
```
fig = px.box(X_train[numeric], title="Four Numeric Columns, Four Different Rulers (957 training animals)")
fig.show()
```

> 🎯 **Predict First:** kNN adds up the differences in all four columns to measure how far apart two animals are. Looking at the box plot, which column will decide who counts as a neighbor?

Task 09A.1 — Raw vs. Scaled *(2 pts)*

Score kNN with `n_neighbors=15` on the four **numeric** columns of the training set, with 5-fold `cross_val_score`:

1. On the raw columns. Store the mean of the 5 scores in `cv_raw`.
2. With a `StandardScaler` in front. Use `make_pipeline(StandardScaler(), KNeighborsClassifier(n_neighbors=15))`: a two-step pipe, so in every round the scaler learns its means and standard deviations from the 4 training folds only. Store the mean score in `cv_scaled`.

The third score is given: kNN on `behavior_score` **alone**, unscaled.

In [ ]:
```
knn = KNeighborsClassifier(n_neighbors=15)

# 1. raw columns
cv_raw = cross_val_score(knn, X_train[numeric], y_train, cv=5).mean()  # SOLUTION

# 2. scaler + kNN, as one pipe
scaled_knn = make_pipeline(StandardScaler(), KNeighborsClassifier(n_neighbors=15))  # SOLUTION
cv_scaled = cross_val_score(scaled_knn, X_train[numeric], y_train, cv=5).mean()  # SOLUTION

cv_behavior = cross_val_score(knn, X_train[["behavior_score"]], y_train, cv=5).mean()
print("four columns, raw:     ", round(cv_raw, 3))
print("four columns, scaled:  ", round(cv_scaled, 3))
print("behavior_score alone:  ", round(cv_behavior, 3))
print("lazy model, 'every animal is adopted':", round(y_train.mean(), 3))
```

In [ ]:
```
grader.check("09A.1")
```

Part 2 — Words to Numbers: One-Hot and Ordinal Encoding (SLO 09A)

kNN needs numbers. Three columns hold words, and they are not all the same kind of word:

- `species` and `intake_type` name **kinds**: a cat is not "between" a dog and a rabbit. They get **one-hot** encoding: one 0/1 column per category
- `size` has a real **order**: Small < Medium < Large < Extra large. **`OrdinalEncoder`** turns ranked words into one column of 0, 1, 2, 3, in the order you give it with `categories=`

Task 09A.2 — Two Encoders *(1 pt)*

Fit both encoders on the **training** set:

1. `onehot`: a `OneHotEncoder()` fit on `X_train[words]`
2. `ordinal`: an `OrdinalEncoder(categories=[sizes])` fit on `X_train[["size"]]`, so the codes follow the sizes from smallest to largest

In [ ]:
```
onehot = OneHotEncoder().fit(X_train[words])  # SOLUTION
ordinal = OrdinalEncoder(categories=[sizes]).fit(X_train[["size"]])  # SOLUTION

print("one-hot columns:", onehot.get_feature_names_out())
print("size codes 0, 1, 2, 3:", ordinal.categories_[0])
```

In [ ]:
```
grader.check("09A.2")
```

Part 3 — One Pipe for Every Step: Pipelines and Grid Search (SLO 09B)

A `ColumnTransformer` sends different columns down different branches, and a `Pipeline` joins it to the model. Everything is fit on training rows only.

Task 09B.1 — Build the Pipeline *(3 pts)*

1. Build a `ColumnTransformer` called `preprocess` with three branches:
   - `("num", StandardScaler(), numeric)`
   - `("size", OrdinalEncoder(categories=[sizes]), ["size"])` . The codes 0–3 already have a standard deviation close to 1, about the same ruler as the scaled columns, so this branch skips the scaler
   - `("words", OneHotEncoder(), words)`
2. Build a `Pipeline` called `model` with two steps: `("prep", preprocess)` and `("knn", KNeighborsClassifier(n_neighbors=15))` .
3. Store the mean 5-fold cross-validation accuracy of `model` on the training set (all of `X_train`) in `cv_pipe` .

In [ ]:
```
# 1. one branch per kind of column
preprocess = ColumnTransformer([  # SOLUTION
# BEGIN SOLUTION NO PROMPT
    ("num", StandardScaler(), numeric),
    ("size", OrdinalEncoder(categories=[sizes]), ["size"]),
    ("words", OneHotEncoder(), words),
])
# END SOLUTION

# 2. preprocessing + kNN
model = Pipeline([("prep", preprocess), ("knn", KNeighborsClassifier(n_neighbors=15))])
# SOLUTION

# 3. cross-validate the whole pipe, on the training set
cv_pipe = cross_val_score(model, X_train, y_train, cv=5).mean()  # SOLUTION

print("cross-validated accuracy:", round(cv_pipe, 3))
model
```

In [ ]:
```
grader.check("09B.1")
```

73.4% with the four numbers only → about 78% with the words added: 4 scaled numbers + 1 size column + 3 species + 3 intake types = 11 columns into kNN. One object holds every step, so in every round the cross-validation refit the scaler and both encoders on 4 folds of the training animals.

Task 09B.2 — Grid Search Over the Whole Pipe *(2 pts)*

Choose k fairly, as at the end of Monday's class: build a `GridSearchCV` on `model` with the grid `{"knn__n_neighbors": ks}`, `cv=5` and `return_train_score=True`. Store it in `grid` and fit it on the **training** set. (The knob's name is the step's name, two underscores, then the knob.)

The rest is given.

In [ ]:
```
ks = [1, 3, 5, 9, 15, 25, 51, 101, 201]
grid = GridSearchCV(model, {"knn__n_neighbors": ks}, cv=5, return_train_score=True)  # SOLUTION
grid.fit(X_train, y_train)  # SOLUTION

best_k = grid.best_params_["knn__n_neighbors"]
best_cv = grid.best_score_
print("best k:", best_k, "| cross-validated accuracy:", round(best_cv, 3))

results = pd.DataFrame(grid.cv_results_)
px.line(results, x="param_knn__n_neighbors", y=["mean_train_score", "mean_test_score"],
markers=True, log_x=True,
        title="Choosing k: Training Folds (train) vs. Cross-Validated (test)").show()
```

In [ ]:
```
grader.check("09B.2")
```

Now, **once**, open the locked test set:

In [ ]:
```
print("best k:", best_k, "| cross-validated:", round(best_cv, 3))
print("test accuracy:", round(grid.score(X_test, y_test), 3))        # refit on all 957 training animals, scored once
print("lazy model, 'every animal is adopted':", round(y_test.mean(), 3))
```

> 📌 **One split:** that test score is on *these* 319 test animals, and only the animals with nothing blank.

Part 4 — The Blanks: Missing Data (SLO 09C)

Parts 1–3 used 1,276 animals. The model never saw the other **724**. Now all 2,000 come back, with the same split recipe:

In [ ]:
```
blank_cols = ["age_years", "weight_kg", "behavior_score"]
print(shelter[blank_cols].isna().sum().to_string())

X = shelter[numeric + ["size"] + words]
y = shelter["adopted"]
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.25, stratify=y, ra
ndom_state=1)
print(len(X_train), "training |", len(X_test), "test")
```

What the shelter staff say about the blanks:

> *Age:* "Ages come from the records that arrive with the animal: from its owner, or from the shelter that sent it. Strays usually come with no records."
>
> *Weight:* "Weights are written on the paper intake form. Some forms were scanned with the bottom edge cut off, and the weight box is at the bottom."
>
> *Behavior:* "We do the temperament assessment during the first week. Staff skip it when an animal is too stressed or reactive to assess safely."

The staff notes are one kind of evidence. The data is another.

Task 09C.1 — Who Has the Blanks? *(2 pts)*

We look for patterns in the **training** animals only: the test animals stay locked, even while we choose a fix.

`blanks` is a True/False table (True = blank). The share of blanks by species is given as an example. Store in `blank_by_intake` the share of blanks by **intake type**. The adoption rates are given.

In [ ]:
```
blanks = X_train[blank_cols].isna()
blank_by_species = blanks.groupby(X_train["species"]).mean()

blank_by_intake = blanks.groupby(X_train["intake_type"]).mean()  # SOLUTION

adoption_if_blank = pd.DataFrame({col: y_train.groupby(blanks[col]).mean() for col in bl
ank_cols})
adoption_if_blank.index = ["adopted, if recorded", "adopted, if blank"]

display(blank_by_species.round(3))
display(blank_by_intake.round(3))
display(adoption_if_blank.round(3))
```

In [ ]:
```
grader.check("09C.1")
```

Task 09C.2 — A Fix for Each Mechanism *(2 pts)*

Write the two fixes; the pipeline around them is given. Each fix is a small pipe of its own (*hint:* `make_pipeline(first_step, second_step)`):

1. `fill_body`: `StandardScaler()`, then `KNNImputer(n_neighbors=5)`. This is fix C, for age (MAR) and weight (MCAR). Scale first, so the imputer's distances aren't all dollars. A `KNNImputer` only sees the columns in its own branch, so this branch also holds age's clues: weight and fee.
2. `flag_behavior`: `SimpleImputer(strategy="median", add_indicator=True)`, then `StandardScaler()`. This is fix D, for behavior (MNAR).

The cross-validation is given. Like every score before it, it uses the training animals only.

In [ ]:
```
fill_body = make_pipeline(StandardScaler(), KNNImputer(n_neighbors=5))  # SOLUTION
flag_behavior = make_pipeline(SimpleImputer(strategy="median", add_indicator=True), Stan
dardScaler())  # SOLUTION

by_mechanism = Pipeline([
    ("prep", ColumnTransformer([
        ("body", fill_body, ["age_years", "weight_kg", "adoption_fee"]),
        ("behavior", flag_behavior, ["behavior_score"]),
        ("size", OrdinalEncoder(categories=[sizes]), ["size"]),
        ("words", OneHotEncoder(), words),
    ])),
    ("knn", KNeighborsClassifier(n_neighbors=best_k)),
])
cv_mechanism = cross_val_score(by_mechanism, X_train, y_train, cv=5).mean()
print("by mechanism, cross-validated:", round(cv_mechanism, 3))
```

In [ ]:
```
grader.check("09C.2")
```

Three Fixes Compared (given — just run it)

A: drop every training animal with a blank, then cross-validate Part 3's pipeline on the rest. **B:** one median for every blank, no indicator. **By mechanism:** your pipeline.

In [ ]:
```
median_everywhere = Pipeline([
    ("prep", ColumnTransformer([
        ("num", make_pipeline(SimpleImputer(strategy="median"), StandardScaler()), numer
ic),
        ("size", OrdinalEncoder(categories=[sizes]), ["size"]),
        ("words", OneHotEncoder(), words),
    ])),
    ("knn", KNeighborsClassifier(n_neighbors=best_k)),
])
keep = X_train.notna().all(axis=1)
cv_drop = cross_val_score(grid.best_estimator_, X_train[keep], y_train[keep], cv=5).mean
()
cv_median = cross_val_score(median_everywhere, X_train, y_train, cv=5).mean()

display(pd.DataFrame({"training animals scored": [keep.sum(), len(X_train), len(X_trai
n)],
                      "CV accuracy": [cv_drop, cv_median, cv_mechanism]},
                     index=["A: drop rows with blanks", "B: one median for every blank",
                            "by mechanism: C for age and weight, D for behavior"]).round
(3))
print("adoption rate, animals kept by A:   ", round(y_train[keep].mean(), 3))
print("adoption rate, animals dropped by A:", round(y_train[~keep].mean(), 3), f"({(~kee
p).sum()} animals)")
```

Now, **once**, open the locked test set:

In [ ]:
```
by_mechanism.fit(X_train, y_train)
not_assessed = X_test["behavior_score"].isna()
print("test accuracy, all", len(X_test), "test animals:", round(by_mechanism.score(X_tes
t, y_test), 3))
print(not_assessed.sum(), "test animals were not assessed:")
print("   predicted adopted:", round(by_mechanism.predict(X_test[not_assessed]).mean(),
3))
print("   actually adopted: ", round(y_test[not_assessed].mean(), 3))
```

> 📌 **One split:** these scores are on *these* 500 test animals, from a shelter we simulated.

The Answer Key: How the Data Was Made (given — just run it)

With real data, nobody hands you the missing values. Here we can peek, because we simulated them.

Adoption. The chance of adoption rises with `behavior_score` and falls with age. Transfers were adopted more, strays less; rabbits, and large and extra-large dogs, less. The fee follows species and age (puppies and kittens cost more), plus noise, so it carries little information of its own.

The blanks:

column	mechanism	how the blanks were made
`weight_kg`	MCAR	7% of animals, at random
`age_years`	MAR	40% of strays, 3% of owner surrenders, 10% of transfers (and strays are younger)
`behavior_score`	MNAR	85% of scores ≤ 3, 35% of 4s, 8% of 5s, 2% of the rest

The file below holds the true value of every blank:

In [ ]:
```
hidden = pd.read_csv("https://cs.calvin.edu/courses/data/202/26fa/datasets/shelter_hidden.csv")      # same animals, same order, no blanks

rows = []
for col in blank_cols:
    blank = shelter[col].isna()
    rows.append({"column": col, "true mean, behind the blanks": hidden.loc[blank, col].mean(),
                 "mean, recorded": hidden.loc[~blank, col].mean()})
display(pd.DataFrame(rows).set_index("column").round(2))
```

- **Behavior (MNAR):** behind the blanks, the true scores average about 3.5, against 6.4 for the assessed animals. The median fill (6) put them among the calm animals. The indicator is what let the model find them again. Without the hidden file, only the staff note could have told us this
- **Age (MAR):** the blank ages belong mostly to strays, who are younger. One median gives each of them 3.2 years. Filling from similar animals (by fee and weight) misses the true age by 1.5 years on average, against 1.9 for the median. The accuracy barely changed, but the filled data is closer to the truth
- **Weight (MCAR):** the blank weights look just like the recorded ones. Any reasonable fix works
- In real data there is no hidden file. That's why the mechanism has to come from **how the data was collected**, not from the scores

Submission

Save your notebook, then upload the **`.ipynb` file** directly to our Moodle assignment page. Don't export, zip, or submit anything else — just the notebook.